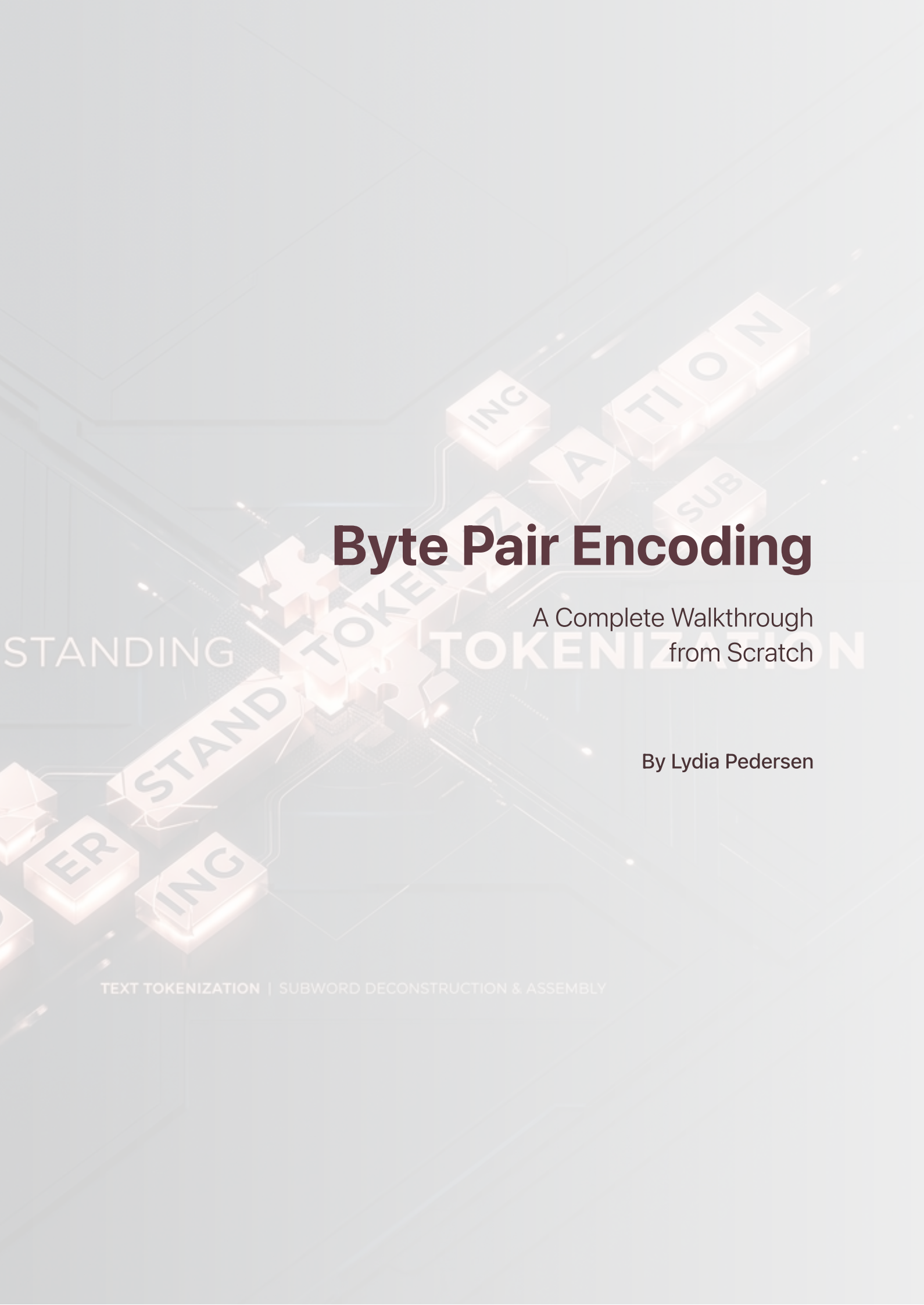


Byte Pair Encoding

A Complete Walkthrough from Scratch

By Lydia Pedersen

Byte Pair Encoding: A Complete Walkthrough from Scratch

Copyright © 2026 Lydia Pedersen. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Table of Contents

Preface: Why Tokenization Matters

Chapter 1: Characters, Words, and the Subword Compromise

Chapter 2: The BPE Training Algorithm

Chapter 3: Training BPE by Hand

Chapter 4: Encoding and Decoding

Chapter 5: From Bytes to Billions

Preface: Why Tokenization Matters

The Transformer architecture relies entirely on continuous vector spaces and linear transformations to process information. Neural networks compute by calculating dot products and multiplying matrices. This strict mathematical reality dictates that raw text cannot be fed directly into the initial embedding matrix. A rigorous translation layer must exist to convert discrete character strings into vectors within a d_{model} -dimensional space before any neural computation can occur.

The Numerical Constraint

Deep learning models require structured mathematical inputs. The attention mechanisms and feed-forward layers detailed throughout the Transformer series execute pure linear algebra. These mathematical operations mandate numerical tensors, which are defined as multi-dimensional arrays of numbers. Tokenization satisfies this requirement by segmenting raw strings into discrete units called tokens.

Each isolated token receives a deterministic integer ID. This ID functions solely as an ordinal lookup index and possesses no inherent mathematical magnitude. The semantic content of a token resides entirely within its corresponding row vector in the embedding matrix W_E . Passing a token ID into the architecture retrieves a dense vector spanning d_{model} dimensions. Every dimension within this vector represents a learned feature of the character string. The numerical weights populating these dimensions are refined and updated throughout the training process to capture semantic relationships.

The quality of the initial string segmentation dictates the foundation upon which all subsequent layers build. A flawed tokenization strategy constrains the resulting geometric vectors, regardless of how effectively the network trains.

The Tokenization Landscape

Several algorithms exist to perform this segmentation, including WordPiece and Unigram language models. Byte Pair Encoding emerges as the dominant standard powering architectures ranging from the GPT family to Llama. The algorithm succeeds by operating without rigid word boundaries, relying instead on a data-driven approach that extracts optimal subword units based strictly on their statistical frequency.

This series explores the mechanics of Byte Pair Encoding from the ground up. To make the abstract process concrete, the algorithm will be executed entirely by hand on a carefully designed toy corpus featuring distinct morphological patterns:

`waking` `woke` `woken` `walking` `walked` `walker` `talking` `talked` `talker`

This vocabulary provides the structural variation necessary to demonstrate how Byte Pair Encoding organically extracts shared stems and suffixes. The corpus groups nine inflected word forms across three verb families spanning present-participle, past-tense, past-participle, and agent-noun variations. Shared linguistic suffixes such as `ing`, `ed`, and `er` guarantee that the algorithm will mathematically discover these recurring morphological structures through statistical frequency alone. The subsequent chapter examines the fundamental tension between character-level and word-level tokenization, proving why the subword compromise is necessary.

Chapter 1: Characters, Words, and the Subword Compromise

The preceding introduction established that neural networks cannot execute mathematical operations on raw text directly. Raw text strings must first map into continuous, d_{model} -dimensional vectors, where every dimension encodes a distinct semantic feature of the string. Each recognized string receives a deterministic, discrete integer ID that indexes its corresponding semantic vector in the embedding matrix W_E . This lookup function necessitates a predefined vocabulary: a finite inventory of recognized textual elements.

The structural definition of these elements dictates two fundamental properties of the neural network. First, the size of this vocabulary defines the strict memory constraints of the model, as every recognized element requires a dedicated vector in memory. Second, the boundaries of these elements define the semantic capabilities of the architecture, as the model can only learn concepts that are cleanly isolated by its vocabulary.

The Word-Level Explosion

Defining the vocabulary space around natural word boundaries presents an intuitive starting point. Under this paradigm, every distinct word in a corpus receives a unique integer ID.

A minimal toy corpus demonstrates the initial viability of this approach. Applying strict word-level boundaries produces a nine-element vocabulary:

`waking` `woke` `woken` `walking` `walked` `walker` `talking` `talked` `talker`

The structural flaw emerges immediately when scaling this strategy to a realistic production corpus. A modern language contains hundreds of thousands of base words. Adding grammatical permutations, prefixes, suffixes, punctuation combinations, and typographical errors results in unbounded expansion.

Every unique token requires a dedicated row in the architecture's embedding matrix W_E , where each scalar entry $w_{i,j}$ stores the learned weight for vocabulary row $i \in \{1, \dots, V\}$ at feature dimension $j \in \{1, \dots, d_{\text{model}}\}$. A vocabulary of one million distinct words mandates an embedding matrix with one million rows:

$$W_E = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,d_{\text{model}}} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,d_{\text{model}}} \\ \vdots & \vdots & \ddots & \vdots \\ w_{V,1} & w_{V,2} & \cdots & w_{V,d_{\text{model}}} \end{bmatrix}_{V \times d_{\text{model}}}$$

An architecture with a vocabulary size V of $1,000,000$ and a model dimensionality d_{model} of $4,096$ contains 4.096×10^9 embedding parameters. Storing this single matrix in 32-bit floating-point precision (4 bytes per parameter) requires $16.384\ \text{GB}$ of memory solely to hold initial token representations. The parameter count scales linearly with V, rendering unconstrained word-level vocabularies computationally intractable. Word-level tokenization also treats `walk` and `walking` as orthogonal indices. This rigid separation discards the morphological relationship shared between the two strings, forcing gradient optimization to encode the underlying geometry of the action twice in unrelated rows.

The Character-Level Destruction

Addressing the dimensionality explosion requires constraining the vocabulary size. The opposite extreme of the tokenization spectrum defines the vocabulary around individual characters.

Applying character-level boundaries to the toy corpus decomposes the text into a compact vocabulary of thirteen distinct elements, including a designated end-of-word marker `</w>`:

`a` `d` `e` `g` `i` `k` `l` `n` `o` `r` `t` `w` `</w>`

This approach completely resolves the memory explosion problem. The embedding matrix shrinks to a negligible size.

The catastrophic failure of this method lies in the semantic destruction of the sequence. Individual characters carry zero inherent meaning. The sequence `t`, `a`, `l`, `k` forces the neural network to expend significant computational depth merely to reconstruct the basic semantic unit of conversation.

The sequence length also expands drastically. The self-attention mechanism within a Transformer scales quadratically with sequence length L, requiring every token to compare itself against every other token in the sequence. Decomposing a single six-character word into six individual tokens increases the pairwise attention cost for that word by a factor of thirty-six (6^2). A short paragraph of text expands into an overwhelmingly long sequence of isolated characters, paralyzing the architecture's ability to process long-range context efficiently.

The Subword Compromise

Resolving the tension between parameter memory and sequence length requires an intermediate granularity that decomposes rare words into reusable morphemes while preserving frequent words as intact tokens:

Granularity	Toy Vocabulary	Production Scale	Embedding Memory	Sequence & OOV Behavior
Word-Level	9 whole words	Unbounded (> 1,000,000)	Extreme (16.384 GB at FP32)	Shortest sequences; fails on unseen words and splits shared stems like `walk` and `walking`
Character-Level	13 symbols (with `</w>`)	Minimal (13 to 256)	Negligible (< 5 MB)	Zero OOV errors, but expands sequence length 5x to 7x and inflates quadratic attention cost
Subword (BPE)	27 tokens (14 merges)	Bounded (32,000 to 128,000)	Balanced (0.5 GB to 2 GB)	Zero OOV errors; keeps frequent words intact while isolating shared suffixes (`ing`, `ed`, `er`)

Decomposing rare and complex words into reusable subword units strictly bounds the overall vocabulary size V, preserving memory efficiency. An encounter with an unfamiliar compound word does not require memorizing a dedicated vector; the architecture instead constructs the representation from common, pre-learned subword pieces. Preserving frequent words as single tokens simultaneously minimizes the overall sequence length L, protecting the attention mechanism from quadratic scaling. A common word remains a single token rather than ten isolated characters, sharply reducing the computational cost of the attention calculation.

Applying a hypothetical subword algorithm to the toy corpus reveals morphological structures hidden across different verb families. The prefixes `walk` and `talk` might remain whole, while common suffixes like `ing` and `ed` separate into independent, reusable tokens.

`walk` `talk` `ing` `ed` `er`

The embedding matrix row corresponding to `ing` captures the geometric representation of continuous action. The architecture effectively constructs composite representations for novel combinations by adding the learned vector for a stem to the learned vector for a suffix.

The Mechanism of Merging

Achieving this ideal subword state requires a deterministic, data-driven mathematical process rather than a manual set of linguistic rules. Byte Pair Encoding accomplishes this through a systematic sequence of merge operations.

The algorithm initializes at the character level, assuming zero inherent lexical meaning. The training loop scans the corpus to identify the single most frequent adjacent pair of tokens and fuses those two discrete elements into a single new token. This merge operation increments the vocabulary size by one while simultaneously shrinking the encoded corpus length. Merging the adjacent character tokens `i` and `n` into the single token `in` adds one entry to the vocabulary while reducing the encoded representation of `walking` by one token. That reduction directly shortens the sequence length that the downstream attention mechanism must process.

This iterative process acts as a localized compression algorithm. The system continuously discovers the most frequent character patterns and binds them together, progressing from individual characters through morphological suffixes, root words, and common phrases.

The next chapter formalizes this training algorithm on the nine-word toy corpus. That derivation specifies the boundary initialization, adjacent-pair frequency counting, and lexicographical tie-breaking rules required to build a subword vocabulary step by step.

Chapter 2: The BPE Training Algorithm

The previous section established that subword tokenization provides the necessary mathematical compromise between the dimensionality explosion of word-level tokenization and the semantic destruction of character-level boundaries. Constructing this precise subword vocabulary requires a data-driven mechanism. Byte Pair Encoding accomplishes this structural transformation through a deterministic, iterative pipeline that binds distinct characters into logical stems and suffixes.

Initialization and Boundary Preservation

The training algorithm begins by isolating the fundamental building blocks of the language. The entire training corpus is first separated into discrete words, typically delineated by whitespace. Every word is then strictly decomposed into a sequence of individual character tokens.

This isolation requires a dedicated mechanism to preserve original word boundaries. Processing the corpus as an unsegmented stream of characters risks merging characters across adjacent words and creating spurious cross-word tokens. Appending a dedicated end-of-word marker, denoted as `</w>`, to the terminal character of every word sequence prevents cross-boundary bleeding.

The toy corpus from the preceding chapter provides a concrete initialization state. The corpus consists of nine distinct inflected word forms, each occurring once. Decomposing these nine words produces a compact initial vocabulary of thirteen character-level tokens:

`a` `d` `e` `g` `i` `k` `l` `n` `o` `r` `t` `w` `</w>`

Each word transforms into a sequence constructed exclusively from this initial character inventory. The string `walking` maps to a sequence of eight discrete elements: `w`, `a`, `l`, `k`, `i`, `n`, `g`, `</w>`. The tokenizer converts the entire corpus into this fully atomized state before any compression occurs.

Pair Frequency Evaluation

The core intelligence of the algorithm relies on discovering structural redundancies within these atomized sequences. The system executes a comprehensive scan over the entire corpus, evaluating every instance where two distinct tokens appear adjacent to one another.

This frequency counting operation tallies the occurrences of every adjacent pair across all words. The goal is to identify the single pair of tokens that co-occur most frequently. In the initialized state of the toy corpus, several character pairings appear repeatedly due to the morphological similarities of the chosen verb families. A tally of the most common adjacent pairs immediately highlights structural patterns:

`a` + `l` →	6 occurrences
`k` + `e` →	6 occurrences
`l` + `k` →	6 occurrences
`w` + `a` →	4 occurrences

The algorithm identifies the pairs `a` + `l`, `k` + `e`, and `l` + `k` as tied for the highest frequency at six occurrences each. A deterministic tie-breaking protocol resolves this collision by selecting the tied pair that sorts first lexicographically. Comparing the three tied pairs, the character `a` precedes `k` and `l` in alphabetical order, dictating the selection of `a` + `l` for the inaugural merge operation.

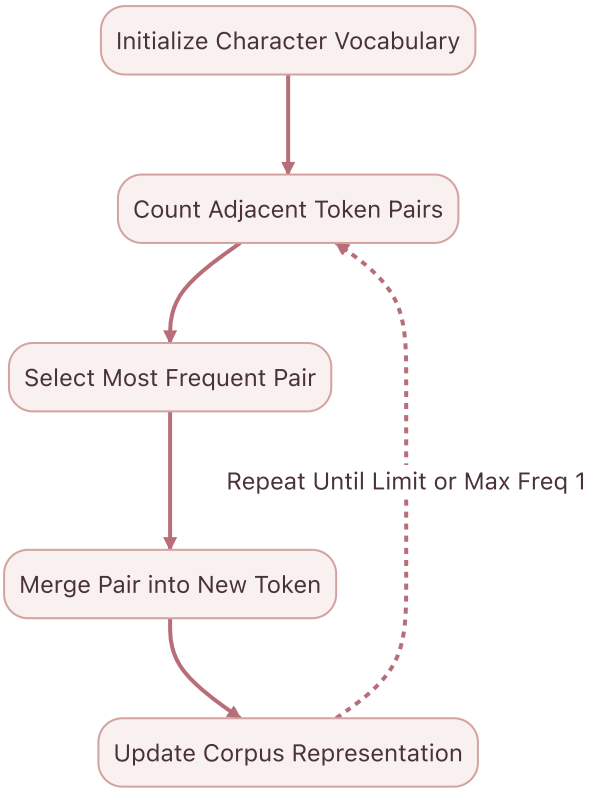
The Greedy Merge Rule

The identification of the highest-frequency pair triggers the central update mechanism. The algorithm formally registers a new token representing the concatenation of the selected pair. This new element is appended to the recognized vocabulary, increasing the total size of the vocabulary space by one.

`a` `d` `e` `g` `i` `k` `l` `n` `o` `r` `t` `w` `</w>` **`al`**

Simultaneously, the algorithm sweeps through the entire corpus and replaces every adjacent occurrence of the individual tokens with the new fused token. This structural update reduces the overall sequence length of the corpus by compressing two discrete tokens into a single unit at every matching location. The sequence representation of the string `walking` compresses from eight elements to seven: `w`, `al`, `k`, `i`, `n`, `g`, `</w>`.

This localized compression logic forms an iterative loop.



Termination Conditions and the Vocabulary-Sequence Tradeoff

The training loop scans the updated corpus, recomputes adjacent pair frequencies, selects the highest-frequency pair, and executes another merge. This cycle repeats until it satisfies one of two termination conditions: reaching a target merge count M, or reaching a state where no adjacent token pair occurs more than once. Every merge operation mints one new token, locking the final vocabulary size V to the initial base vocabulary count $|V_0|$ plus the number of executed merges M ($V = |V_0| + M$). Production language models typically execute between $32,000$ and $128,000$ merge iterations to construct their final vocabulary.

Selecting the target merge budget M balances two competing computational scaling laws:

- **Linear Embedding Parameter Cost ($\mathcal{O}(V \cdot d_{\text{model}})$):** Every merged token adds one row of width d_{model} to the embedding matrix W_E, increasing static parameter memory linearly with vocabulary size V.
- **Quadratic Self-Attention Compute Cost ($\mathcal{O}(L^2 \cdot d_{\text{model}})$):** Every merge shortens the encoded sequence length L across the corpus, quadratically reducing the pairwise token comparisons required inside self-attention layers.

Greedily merging the most frequent adjacent pairs first maximizes sequence compression per added vocabulary row. The algorithm systematically binds individual characters into morphological suffixes, root stems, and whole words.

The next chapter executes this training procedure by hand across all fourteen merge steps on the nine-word toy corpus. Tracing the pair frequency counts and corpus updates at each iteration demonstrates how statistical compression isolates stems such as `alk` and `woke` alongside suffixes such as `ing</w>` without explicit linguistic rules.

Chapter 3: Training BPE by Hand

The formal definition of the Byte Pair Encoding algorithm establishes a greedy, iterative mechanism for subword compression. Executing this mechanism against a concrete corpus demonstrates how these abstract rules operate in practice. The procedure relies entirely on statistical frequency rather than linguistic programming.

The Initial State

The training process begins with the nine distinct inflected word forms defined in the preceding chapter. Each word splits into individual character tokens terminated by the `</w>` boundary marker. This initialized corpus state allows the algorithm to track discrete token sequences and their absolute frequencies across the text.

`w` `a` `k` `i` `n` `g` `</w>`
`w` `o` `k` `e` `</w>`
`w` `o` `k` `e` `n` `</w>`
`w` `a` `l` `k` `i` `n` `g` `</w>`
`w` `a` `l` `k` `e` `d` `</w>`
`w` `a` `l` `k` `e` `r` `</w>`
`t` `a` `l` `k` `i` `n` `g` `</w>`
`t` `a` `l` `k` `e` `d` `</w>`
`t` `a` `l` `k` `e` `r` `</w>`

The First Merges: Deriving Roots

The algorithm evaluates the frequency of every adjacent token pair in this initial state. The pairs `a` + `l`, `k` + `e`, and `l` + `k` tie for the highest frequency with six occurrences each. Lexicographical sorting dictates the selection of `a` + `l` for the inaugural merge operation. The new token `al` is formally registered, and the corpus representation is updated globally.

Step 1	`a` + `l`	→ `al`	6 occurrences

w a k i n g </w>
w o k e </w>
w o k e n </w>
w al k i n g </w>
w al k e d </w>
w al k e r </w>
t al k i n g </w>
t al k e d </w>
t al k e r </w>

This merge operation immediately alters the subsequent frequency distribution. The second merge iteration tallies adjacent pairs across the updated corpus, where `al` + `k` and `k` + `e` tie for the highest frequency at six occurrences each. Lexicographical tie-breaking selects `al` + `k` as the next merge target.

Step 2 **al** + k → **alk** 6 occurrences

w a k i n g </w>
w o k e </w>
w o k e n </w>
w alk i n g </w>
w alk e d </w>
w alk e r </w>
t alk i n g </w>
t alk e d </w>
t alk e r </w>

The merge operation fuses these tokens, creating the cohesive `alk` unit. The third merge iteration establishes that the pair `alk` + `e` occurs four times, binding the root verb to the start of its suffixes.

Step 3 **alk** + e → **alke** 4 occurrences

Three merge iterations suffice for the counting mechanism to derive `alk`, the central morphological root shared by two-thirds of the corpus. This fragment serves as the structural foundation for *walking*, *walked*, *walker*, *talking*, *talked*, and *talker*, compressing the internal representation of the text.

w a k i n g </w>
w o k e </w>
w o k e n </w>
w alk i n g </w>
w alke d </w>
w alke r </w>
t alk i n g </w>
t alke d </w>
t alke r </w>

Assembling Suffixes

The subsequent three merge iterations reveal how boundary markers influence the derivation of suffixes. Iteration four evaluates the updated sequences and identifies the highest frequency pair ending the gerund verbs.

Step 4 `g` + `</w>` → **`g</w>`** 3 occurrences

Iteration five targets the interior of the suffix, binding the preceding characters.

Step 5 `i` + `n` → **`in`** 3 occurrences

Iteration six combines these two newly formed tokens.

Step 6 **`in`** + **`g</w>`** → **`ing</w>`** 3 occurrences

This sequence formally registers the morphological `-ing` suffix into the subword vocabulary. The token boundary marker `</w>` guarantees that this newly minted token `ing</w>` specifically represents the suffix at the end of a word, preventing it from incorrectly matching the substring "ing" in the middle of an unrelated word.

Accelerated Convergence

The iterative compression rapidly integrates the remaining structure. The next five iterations bind the derived roots and suffixes into complete semantic units.

Step 7	`alk` + `ing</w>`	→ `alking</w>`	2 occurrences
Step 8	`alke` + `d`	→ `alked`	2 occurrences
Step 9	`alke` + `r`	→ `alker`	2 occurrences
Step 10	`alked` + `</w>`	→ `alked</w>`	2 occurrences
Step 11	`alker` + `</w>`	→ `alker</w>`	2 occurrences

The initial `alk` fragment transforms into three complete lexical structures: `alking</w>`, `alked</w>`, and `alker</w>`. The counting procedure then evaluates the remaining verb family.

Step 12	`k` + `e`	→ `ke`	2 occurrences
Step 13	`o` + `ke`	→ `oke`	2 occurrences
Step 14	`w` + `oke`	→ `woke`	2 occurrences

Fourteen merge operations transform the original sequences of isolated characters into consolidated subword fragments. The internal representation of the text achieves substantial sequence compression.

`w` `a` `k` `ing</w>`

`woke` `</w>`

`woke` `n` `</w>`

`w` `alking</w>`

`w` `alked</w>`

`w` `alker</w>`

`t` `alking</w>`

`t` `alked</w>`

`t` `alker</w>`

Finalization of the Vocabulary

The Byte Pair Encoding algorithm halts when the maximum frequency of every remaining adjacent pair drops to `1`, as merging a pair that occurs only once provides no generalizable compression across words. Step 14 reduces the frequency of every remaining adjacent pair in the toy corpus to `1`, terminating the training loop.

Executing these 14 merge operations produces a deterministic, compressed token inventory. A structural distinction separates the final encoded corpus from the trained vocabulary dictionary. The nine sequences displayed above represent the compressed state of the training corpus, where repeated character patterns have collapsed into larger subword units.

The encoded corpus does not collapse into nine single-token words. This partial compression reflects a core design property of subword tokenization: the word *talking* appears only once in the corpus, so its leading consonant `t` never reaches the pair frequency required to merge with `alking</w>`. Rare words remain represented as compositions of smaller, shared subwords.

The trained *vocabulary dictionary* contains 27 tokens: the 13 initial base characters plus the 14 subword tokens minted across the merge sequence. This vocabulary count directly governs the input dimensions of the neural network, instantiating the embedding matrix with 27 rows corresponding to the 27 learned tokens.

`</w>` `a` **`al`** **`alk`** **`alke`** **`alked`** **`alked</w>`** **`alker`** **`alker</w>`** **`alking</w>`** `d` `e` `g` **`g</w>`**
`i` **`in`** **`ing</w>`** `k` **`ke`** `l` `n` `o` **`oke`** `r` `t` `w` **`woke`**

The statistical frequency analysis identified the recurring morphological regularities of the corpus. This trained 27-token vocabulary now stands as a fixed, immutable dictionary. The next chapter explores how this finalized dictionary processes unseen text through the corresponding encoding algorithm.

Chapter 4: Encoding and Decoding

The 14-step training algorithm halted after constructing a 27-token vocabulary built upon the recurring morphological patterns of the corpus. The resulting list of learned merge rules now stands as a static, ordered mapping protocol. Deploying this protocol to tokenize unseen text forms the next phase of the subword pipeline.

The Ordered Merge Algorithm

The encoding process abandons the dynamic frequency counting that drove the training phase. The encoder instead applies the fixed sequence of merge operations acquired during training, executing those rules on incoming text strictly in the chronological order they were learned.

This sequential application constitutes a greedy, longest-match algorithm. Enforcing the original chronological order of merges guarantees that the resulting tokenization mathematically reflects the most dominant statistical patterns of the training data. The algorithm begins by splitting the incoming text into individual characters and appending the boundary marker `</w>`.

Encoding the Known Distribution

Applying this static protocol to words from the original training distribution yields compact subword representations. Encoding the word `walked`, which appears in the original training corpus, demonstrates the systematic execution of these ordered rules.

Initial	`w` `a` `l` `k` `e` `d` `</w>`	
Step 1	`a` + `l` → `w` **`al`** `k` `e` `d` `</w>`	(Rule 1)
Step 2	**`al`** + `k` → `w` **`alk`** `e` `d` `</w>`	(Rule 2)
Step 3	**`alk`** + `e` → `w` **`alke`** `d` `</w>`	(Rule 3)
Step 4	**`alke`** + `d` → `w` **`alked`** `</w>`	(Rule 8)
Step 5	**`alked`** + `</w>` → `w` **`alked</w>`**	(Rule 10)

The algorithm completely ignores rules 4, 5, 6, 7, and 9 as the necessary adjacent pairs do not exist in the current sequence. The word systematically collapses down to two final tokens: `w` and `alked</w>`.

A similar execution dictates the encoding of `waking`, which is also present in the original training corpus.

Initial	`w` `a` `k` `i` `n` `g` `</w>`	
Step 1	`g` + `</w>` → `w` `a` `k` `i` `n` **`g</w>`**	(Rule 4)
Step 2	`i` + `n` → `w` `a` `k` **`in`** **`g</w>`**	(Rule 5)
Step 3	**`in`** + **`g</w>`** → `w` `a` `k` **`ing</w>`**	(Rule 6)

The training process naturally halted before discovering a sequence unifying `w`, `a`, and `k`, causing the final encoding to remain fractured across four distinct tokens: `w`, `a`, `k`, and `ing</w>`.

Subword Generalization on Novel Text

The generalization capability of the Byte Pair Encoding algorithm emerges when encoding text absent from the training corpus. The encoder reuses learned morphological patterns to parse unseen words into known subword pieces.

Processing the novel word `stalking`, which does not appear in the training corpus, demonstrates this decomposition mechanism. The encoder evaluates the string against the identical ordered rule sequence.

Initial	`s` `t` `a` `l` `k` `i` `n` `g` `</w>`	
Step 1	`a` + `l` → `s` `t` **`al`** `k` `i` `n` `g` `</w>`	(Rule 1)
Step 2	**`al`** + `k` → `s` `t` **`alk`** `i` `n` `g` `</w>`	(Rule 2)
Step 3	`g` + `</w>` → `s` `t` **`alk`** `i` `n` **`g</w>`**	(Rule 4)
Step 4	`i` + `n` → `s` `t` **`alk`** **`in`** **`g</w>`**	(Rule 5)
Step 5	**`in`** + **`g</w>`** → `s` `t` **`alk`** **`ing</w>`**	(Rule 6)
Step 6	**`alk`** + **`ing</w>`** → `s` `t` **`alking</w>`**	(Rule 7)

The ordered merge pass leaves the unfamiliar prefix consonants `s` and `t` unmerged as individual characters while assembling the frequent suffix `alking</w>` learned from the training data. Provided the base vocabulary includes the full lowercase alphabet, the resulting three-token sequence `s`, `t`, and `alking</w>` decomposes the unseen word into existing character and subword units.

The novel word `awoke`, similarly absent from the original corpus, undergoes an identical decomposition.

Initial	`a` `w` `o` `k` `e` `</w>`	
Step 1	`k` + `e` → `a` `w` `o` **`ke`** `</w>`	(Rule 12)
Step 2	`o` + **`ke`** → `a` `w` **`oke`** `</w>`	(Rule 13)
Step 3	`w` + **`oke`** → `a` **`woke`** `</w>`	(Rule 14)

The algorithm discovers the familiar sequence `woke`, leaving the independent prefix `a` and the final boundary marker `</w>` as separate units.

The Triviality of Decoding

Reversing this process to recover raw text from a token sequence requires no complex algorithms or stored merge rules. Decoding acts as a direct string concatenation operation. The decoder joins the sequence of string tokens together and replaces each `</w>` boundary marker with a standard whitespace character. This inverse mapping recovers the original text with full fidelity.

The character-level tokenization pipeline transforms variable-length text into discrete token sequences, yet the dictionary remains bound to its initial character inventory. Restricting the base vocabulary strictly to the 13 characters observed in the toy corpus causes the encoder to fail whenever an input contains an unobserved character such as the leading `s` in `stalking`, an uppercase letter, or a punctuation mark. The dictionary lacks a token ID for the unknown symbol, triggering an out-of-vocabulary error. The final chapter eliminates this limitation by driving Byte Pair Encoding down to the raw UTF-8 byte level to achieve universal text encoding.

Chapter 5: From Bytes to Billions

The character-level encoding pipeline relies on a fragile assumption by limiting its initial base vocabulary to the characters observed in the training corpus. Encountering a novel symbol during inference, such as an emoji, a non-Latin script character, or an unobserved punctuation mark, drives the greedy longest-match algorithm into a dead end. The encoder cannot match the unknown character to any existing token ID, triggering an out-of-vocabulary failure state that halts processing.

The Universal Byte-Level Foundation

Resolving this failure state requires replacing character vocabularies with raw binary units. Digital hardware stores data as bits representing binary states of `1` or `0`, and operating systems group those bits into standardized eight-bit blocks called *bytes*. An eight-bit byte spans $2^8 = 256$ distinct binary permutations, establishing a fixed integer range from `0` through `255`.

Every text symbol, regardless of script or complexity, serializes into a sequence of bytes under the UTF-8 standard. This encoding standard provides a finite, universal foundation for tokenization. Production tokenizers initialize their base vocabulary with the 256 raw byte values rather than an unpredictable inventory of human-readable characters.

A critical distinction governs how the algorithm interacts with these values. Byte Pair Encoding does not operate at the bit level and possesses no concept of the underlying ones and zeros. The algorithm treats each integer value from `[0]` to `[255]` as a single, indivisible atomic token.

The ASCII Range and Beyond

Initializing the vocabulary with this 256-token foundation guarantees that every valid string can be encoded. Standard encoding tables exhibit a structural boundary at integer `[127]`, which corresponds to `7F` in hexadecimal. The first `128` integer values (`0` through `127`) represent the classic ASCII character set covering English letters, digits, and basic punctuation. Those `128` characters fit inside a single 8-bit byte, mapping the lowercase letter `w` to the single byte integer `119`.

The broader Unicode standard defines over one million symbols across global scripts. UTF-8 accommodates this scale while preserving backward compatibility with 1-byte ASCII systems by operating as a *variable-length* encoding. Characters outside the basic ASCII range serialize into sequences of two to four bytes: the accented character `é` maps to the two-byte sequence `[195, 169]`, and the rocket emoji 🚀 maps to the four-byte sequence `[240, 159, 154, 128]`.

The tokenizer itself never inspects raw binary bits. A host runtime such as Python first executes a standard `encode("utf-8")` call to translate an input string into an array of byte integers. Typing `é` produces the integer array `[195, 169]`, which the runtime passes directly to the tokenizer. Display rendering engines concatenate the underlying bits of that multi-byte sequence to draw a single glyph on screen, whereas the Byte Pair Encoding algorithm operates strictly on the discrete integer sequence.

The Unicode Dimensionality Problem

The existence of over one million pre-assigned Unicode code points suggests an alternative design: initializing the base vocabulary directly with all 1.1 million Unicode characters and assigning merged combinations such as `[x]` + `[y]` → `[z]` to token IDs starting above `1,114,112`. Capping the base vocabulary at `[255]` and learning abstract entries from `[256]` onward is dictated by matrix parameter efficiency.

The embedding matrix of a Transformer allocates one row vector for every token in the vocabulary. Initializing the base vocabulary with all 1.1 million Unicode code points would instantiate more than one million embedding rows before a single subword merge occurred, dedicating billions of parameters to rare scripts and symbols that scarcely appear in the training distribution.

Capping the base vocabulary at the 256 raw bytes restricts the foundational embedding cost to 256 rows. This compact base allocation preserves the model parameter budget so the training algorithm can dedicate a 50,000-token vocabulary entirely to byte combinations that recur frequently in the training corpus.

Processing the UTF-8 array for `é` presents the tokenizer with the integer `[195]` followed by `[169]`. Those two byte values co-occur whenever `é` appears in the corpus, causing the BPE frequency tally to rank `[195, 169]` as a high-frequency pair and fuse it into a single abstract token ID (`[195]` + `[169]` → `[257]`). Scaling this byte-level merge loop across large datasets fuses multi-byte characters, morphological suffixes such as `i` `n` `g`, and whole words such as `t` `h` `e` into single token IDs within a compact, data-driven vocabulary.

Decoding and the UTF-8 Guarantee

The restriction to abstract integers necessitates a dedicated text reconstruction protocol. Transforming the abstract token `[257]` back into the printable character `é` requires reversing the merge operations.

The tokenizer maintains a lookup table of every merge rule learned during training. During the decoding phase, when the neural network emits token `[257]`, the decoder references this table to invert the merge, expanding `[257]` back into its constituent byte pair `[195, 169]` and returning the raw byte array to the host language.

Reconstructing characters from a flat byte array requires an unambiguous framing rule to determine whether `[195, 169]` represents the single two-byte character `é` or two separate single-byte characters (`Ã` followed by `©`). The UTF-8 specification resolves this ambiguity by partitioning bytes into **Start Bytes** and **Continuation Bytes** using dedicated binary bit prefixes:

Binary Bit Prefix	Byte Classification	Sequence Length	Framing Behavior
`0xxxxxxx`	**ASCII Byte**	1 byte	Encodes a standalone 7-bit ASCII character (`0` to `127`)
`110xxxxx`	**2-Byte Start Byte**	2 bytes	Opens a two-byte code point followed by one continuation byte
`1110xxxx`	**3-Byte Start Byte**	3 bytes	Opens a three-byte code point followed by two continuation bytes
`11110xxx`	**4-Byte Start Byte**	4 bytes	Opens a four-byte code point followed by three continuation bytes
`10xxxxxx`	**Continuation Byte**	Trailing byte	Carries payload bits immediately following a multi-byte start byte

Every trailing byte inside a multi-byte character carries the `10` binary prefix. The integer `195` equals `11000011` in binary, marking a two-byte Start Byte, while `169` equals `10101001` in binary, marking a Continuation Byte. A UTF-8 text engine cannot misinterpret `[195, 169]` as two separate characters because a `10xxxxxx` continuation byte is syntactically invalid as a standalone character.

This prefix structure makes a UTF-8 byte stream self-synchronizing. A parser that indexes into the middle of a byte array and reads a `10xxxxxx` prefix immediately detects an interior continuation byte, scans backward to locate the governing Start Byte, and reads the declared number of bytes forward.

The Genius of Partial Merges

The strict limitation of merging only adjacent pairs prevents the algorithm from compressing a 3-byte or 4-byte character in a single operation.

Encountering a 3-byte character, such as a Hindi Devanagari symbol represented by `[224]` `[164]` `[185]`, forces the tokenizer to merge the sequence iteratively. First, it will merge `[224]` + `[164]` into a new abstract token like `[350]`. Later, it will merge `[350]` + `[185]` into `[412]`.

Leaving a character partially merged as `[350]` and `[185]` poses zero risk of corrupting the text.

Tokens are never decoded in isolation. The neural network emits a sequence of token IDs, and the tokenizer expands the entire sequence into a contiguous byte array before invoking the UTF-8 decoder. Whether those bytes originated from one merged token or three separate tokens has no effect on decoding: the presence of `[224, 164, 185]` in the output stream directs the Start Byte (`224`, binary `11100000`) to consume all three bytes and render the Devanagari character.

Partial merges also provide a structural compression advantage across non-Latin scripts. The Unicode standard groups characters from the same writing system into contiguous code point blocks, causing most Hindi Devanagari characters to share the leading byte pair `[224, 164]`. Merging those first two bytes into a single token `[350]` establishes a shared script-prefix token. The model can then represent individual Devanagari characters as a two-token composition (`[Devanagari_Prefix]` + `[Terminal_Byte]`) without allocating thousands of separate whole-character entries.

Re-Training the Toy Example in Bytes

To prove that the mechanical logic remains mathematically identical, the toy corpus is completely detached from human-readable characters and converted into raw integers.

A dedicated boundary token, `</w>`, marks the end of each word. The boundary marker `</w>` does not serialize into its literal four-character ASCII byte sequence (`<` , `/` , `w` , `>`). This marker functions as an abstract control token injected by the pre-tokenizer rather than literal text from the document. Production tokenizers prevent control-token collisions with raw text by assigning control tokens to integer IDs above the `0` through `255` byte range, reserving `[256]` for `</w>` and minting merged subword tokens starting at `[257]` .

INITIAL CORPUS:

w a k i n g </w>	[119] [97] [107] [105] [110] [103] [256]
w o k e </w>	[119] [111] [107] [101] [256]
w o k e n </w>	[119] [111] [107] [101] [110] [256]
w a l k i n g </w>	[119] [97] [108] [107] [105] [110] [103] [256]
w a l k e d </w>	[119] [97] [108] [107] [101] [100] [256]
w a l k e r </w>	[119] [97] [108] [107] [101] [114] [256]
t a l k i n g </w>	[116] [97] [108] [107] [105] [110] [103] [256]
t a l k e d </w>	[116] [97] [108] [107] [101] [100] [256]
t a l k e r </w>	[116] [97] [108] [107] [101] [114] [256]

Step 1 a + l → **al** [97] + [108] → **[257]**

w a k i n g </w>	[119] [97] [107] [105] [110] [103] [256]
w o k e </w>	[119] [111] [107] [101] [256]
w o k e n </w>	[119] [111] [107] [101] [110] [256]
w **al** k i n g </w>	[119] **[257]** [107] [105] [110] [103] [256]
w **al** k e d </w>	[119] **[257]** [107] [101] [100] [256]
w **al** k e r </w>	[119] **[257]** [107] [101] [114] [256]
t **al** k i n g </w>	[116] **[257]** [107] [105] [110] [103] [256]
t **al** k e d </w>	[116] **[257]** [107] [101] [100] [256]
t **al** k e r </w>	[116] **[257]** [107] [101] [114] [256]

Step 2 al + k → **alk** [257] + [107] → **[258]**

w a k i n g </w>	[119] [97] [107] [105] [110] [103] [256]
w o k e </w>	[119] [111] [107] [101] [256]
w o k e n </w>	[119] [111] [107] [101] [110] [256]
w **alk** i n g </w>	[119] **[258]** [105] [110] [103] [256]
w **alk** e d </w>	[119] **[258]** [101] [100] [256]
w **alk** e r </w>	[119] **[258]** [101] [114] [256]
t **alk** i n g </w>	[116] **[258]** [105] [110] [103] [256]
t **alk** e d </w>	[116] **[258]** [101] [100] [256]
t **alk** e r </w>	[116] **[258]** [101] [114] [256]

⋮

The byte-level training loop halts at the same 14-step boundary as the character-based model. The identical subword hierarchy emerges because the underlying pair-frequency counts remain invariant under integer relabeling. Operating on raw UTF-8 bytes preserves morphological discovery while eliminating out-of-vocabulary failures.

The Embedding Matrix Connection

Vocabulary Size and Sequence Scaling

Establishing a 256-byte foundation allows the training algorithm to scale across multi-gigabyte corpora. Production tokenizers such as SentencePiece and `tiktoken` execute this greedy pair-counting and merging loop tens of thousands of times across large training datasets.

The configured number of merge iterations determines the final vocabulary size, V. A small vocabulary leaves words fragmented into short subpieces, lengthening the token sequences that the quadratic self-attention mechanism must process. A larger vocabulary consolidates frequent words into single token IDs such as `[270]`, shortening sequence lengths while adding one embedding row per new token. Production language models balance sequence compression against embedding memory by targeting a vocabulary size V between `32,000` and `100,000` tokens.

Embedding Matrix Lookup Geometry

The vocabulary cardinality V defines the input dimension of the Transformer network. The initial step of a Transformer forward pass maps each discrete token ID into a continuous vector space through the embedding matrix, denoted as W_E.

The embedding matrix W_E functions as a parameterized coordinate lookup table containing one row vector for each of the V tokens in the vocabulary. The width of each row matches the hidden state dimensionality of the model, d_{model}, locking the matrix dimensions to $V \times d_{\text{model}}$:

$$W_E = \begin{bmatrix} \text{Coordinate vector for token } [0] \\ \text{Coordinate vector for token } [1] \\ \text{Coordinate vector for token } [2] \\ \vdots \\ \text{Coordinate vector for token } [V-1] \end{bmatrix}_{V \times d_{\text{model}}}$$

Training a Byte Pair Encoding tokenizer to a vocabulary size of $V = 50,257$ tokens for a Transformer with hidden dimensionality $d_{\text{model}} = 768$ instantiates an embedding matrix with $50,257 \times 768 = 38,597,376$ learnable parameters.

This dimensional coupling completes the input transformation pipeline: raw text serializes into UTF-8 bytes, ordered BPE merge rules fuse frequent byte sequences into subword tokens, the vocabulary dictionary maps each token to an integer index, and that integer index selects the corresponding row vector from W_E. The companion Embeddings series examines how gradient descent optimizes those continuous coordinate vectors to encode semantic relationships.